

```

    return i;
}

vector<int> quicksort (vector<int> &arr, int lb, int ub) {
    if (lb < ub) {
        int mid = partition(arr, lb, ub);
        mid-1 - lb+1
         $\Rightarrow T(\text{mid} - \text{lb})$ 
        quicksort(arr, lb, mid-1);
        quicksort(arr, mid+1, ub);
    }
     $\hookrightarrow$  similarly  $T(\text{ub} - \text{mid})$ 
    return arr;
}

```

Recurrence Relation

$$T(n) \begin{cases} c \\ T(\text{mid} - \text{lb}) + T(\text{ub} - \text{mid}) + n; n > 1 \end{cases}$$

$\nearrow$ from partitioning procedure

Best case / Average case

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

$$\Rightarrow \Theta(n \log n)$$

Worst Case Scenario

$$T(n) = T(n-1) + 1 + n;$$

$$T(n) = T(n-1) + n;$$

$$T(n) = O(n^2)$$

50, 70, 60, 69, 72, 87, 68

i j → j → j → j → j → j
p

() 50 (70, 60, 69, 72, 87, 68) ~ Worst Case.
↳ empty QuickSort(arr, 1, 6)

The Last Algorithm to use if the array is almost completely sorted array.

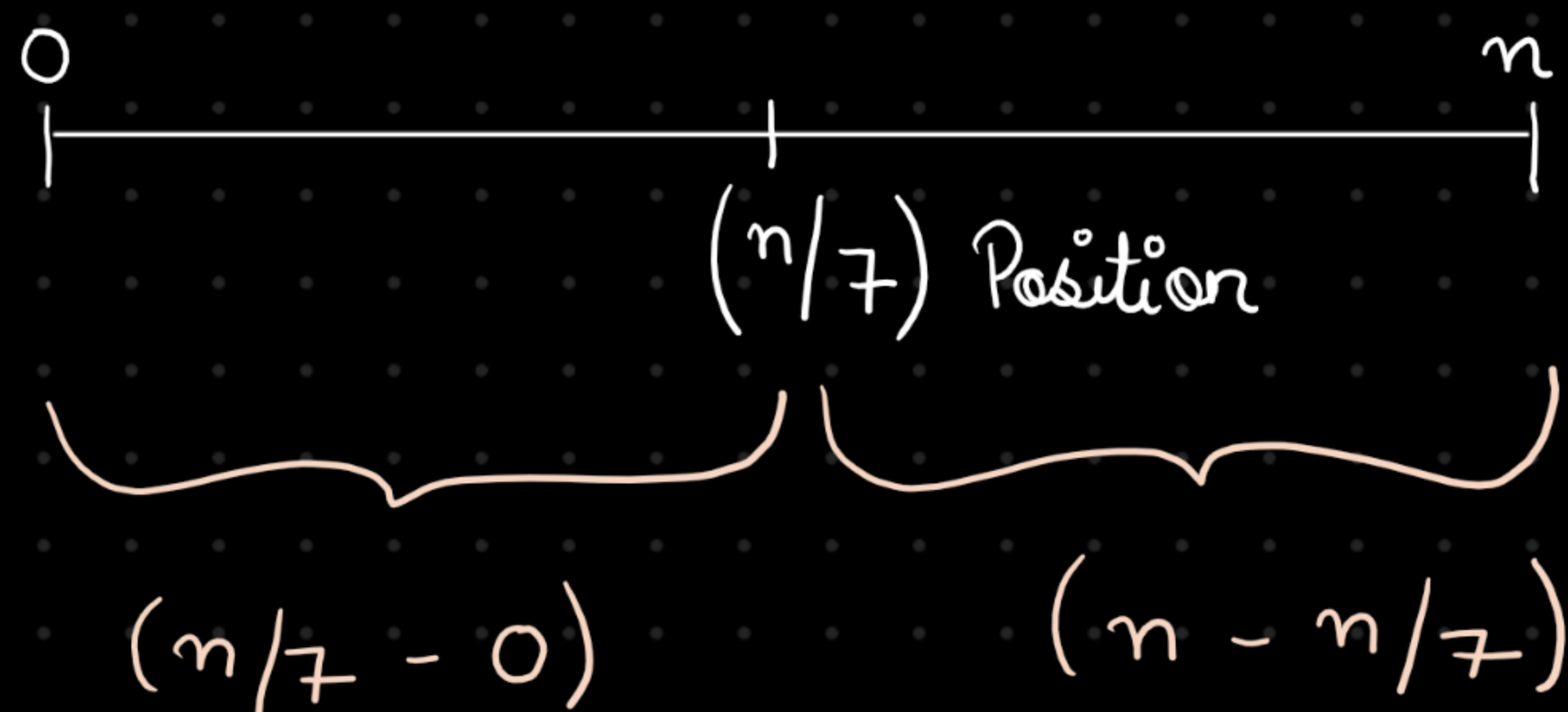
A better Quicksort Method

↳ Randomized Quicksort
↙

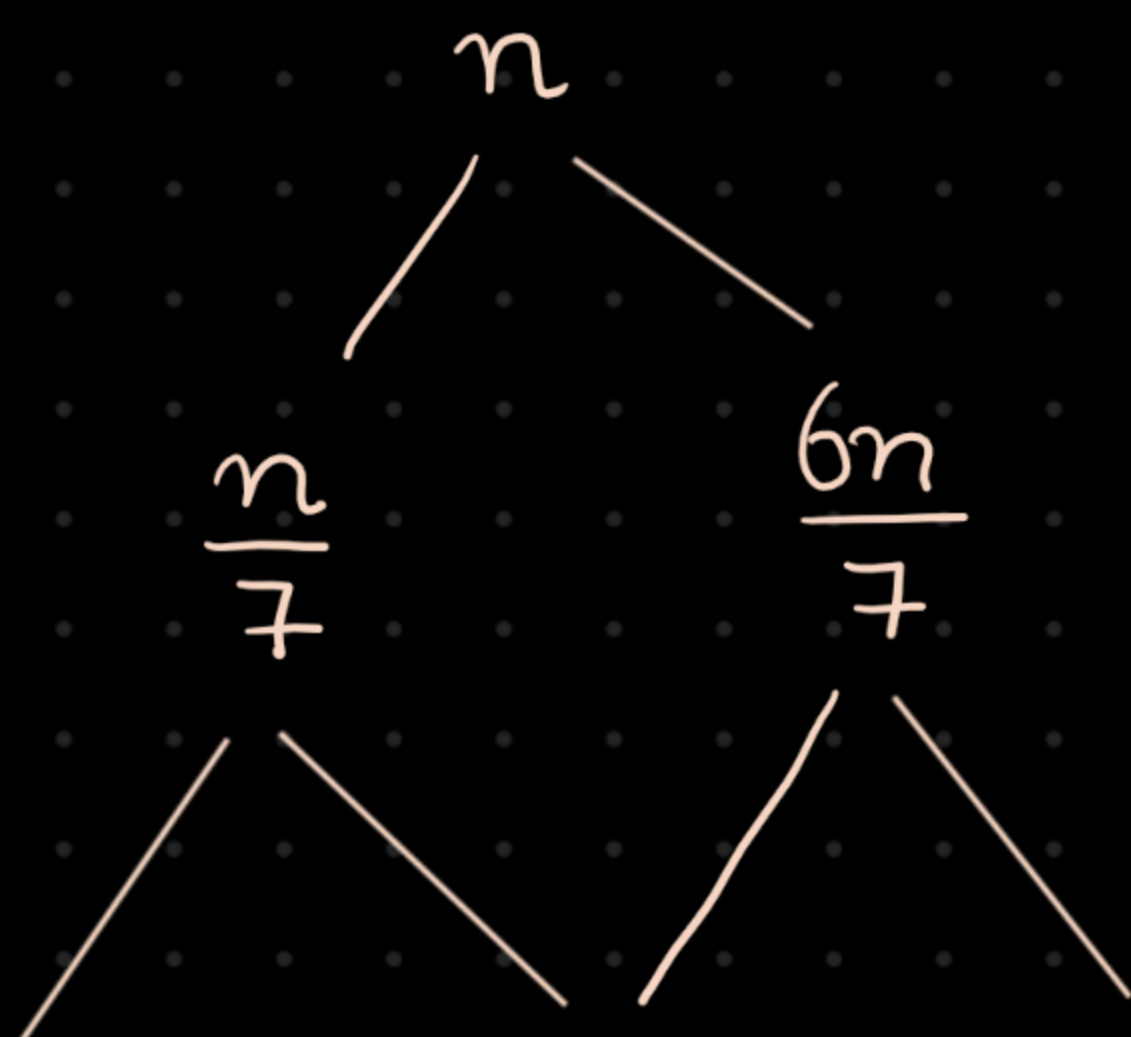
Choose pivot element randomly to decrease the probability of worst case scenario.

Interview

- 1) $n/7^{\text{th}}$ smallest element $\rightarrow$ Quicksort + Partition
 $\hookrightarrow O(n^2)$ $\hookrightarrow O(n)$



$$T(n) = T(n/7) + n(6n/7) + n^2 + n$$



- 2) K^{th} smallest element

(20, 47, 52, 12, 26, 69, 74)

$K = 2$

$\hookrightarrow 2^{\text{nd}}$ smallest element

Selection

Procedure

$\hookrightarrow \text{pivot} + 1 (\text{position})$

selectionProcedure(arr, p, q):

if $K == n$ $\hookrightarrow$ returned by partition Algo

Quicksort → (Doesn't need the merge procedure)

↳ Inplace

↳ unstable (Does not guarantee preserving the relative Order of equal elements).

~~45~~, 40, ~~70~~, ~~10~~, ~~30~~, 90, ~~45~~, 67, 79
P J → J → J → J → J → J → J → J
I → I → I → I → I

↳ 45, 40, 10, 30, 50, 90, 70, 67, 79 ~ partition
less than pivot correct position greater than pivot

```
int partition(vector<int> &arr, int lb, int ub) {  
    int pivot = arr[lb];  
    int i = pivot;  
    for (int j = i + 1; j <= ub; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
    swap(arr[i], arr[lb]);  
}
```



```

    return i;
}

vector<int> quicksort (vector<int> &arr, int lb, int ub) {
    if (lb < ub) {
        int mid = partition(arr, lb, ub);
        mid-1 - lb+1
         $\Rightarrow T(\text{mid} - \text{lb})$ 
        quicksort(arr, lb, mid-1);
        quicksort(arr, mid+1, ub);
    }
     $\hookrightarrow$  similarly  $T(\text{ub} - \text{mid})$ 
    return arr;
}

```

Recurrence Relation

$$T(n) \begin{cases} c \\ T(\text{mid} - \text{lb}) + T(\text{ub} - \text{mid}) + n; n > 1 \end{cases}$$

$\nearrow$ from partitioning procedure

Best case / Average case

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

$$\Rightarrow \Theta(n \log n)$$

Worst Case Scenario

$$T(n) = T(n-1) + 1 + n;$$

$$T(n) = T(n-1) + n;$$

$$T(n) = O(n^2)$$

50, 70, 60, 69, 72, 87, 68

i j → j → j → j → j → j
p

() 50 (70, 60, 69, 72, 87, 68) ~ Worst Case.
↳ empty QuickSort(arr, 1, 6)

The Last Algorithm to use if the array is almost completely sorted array.

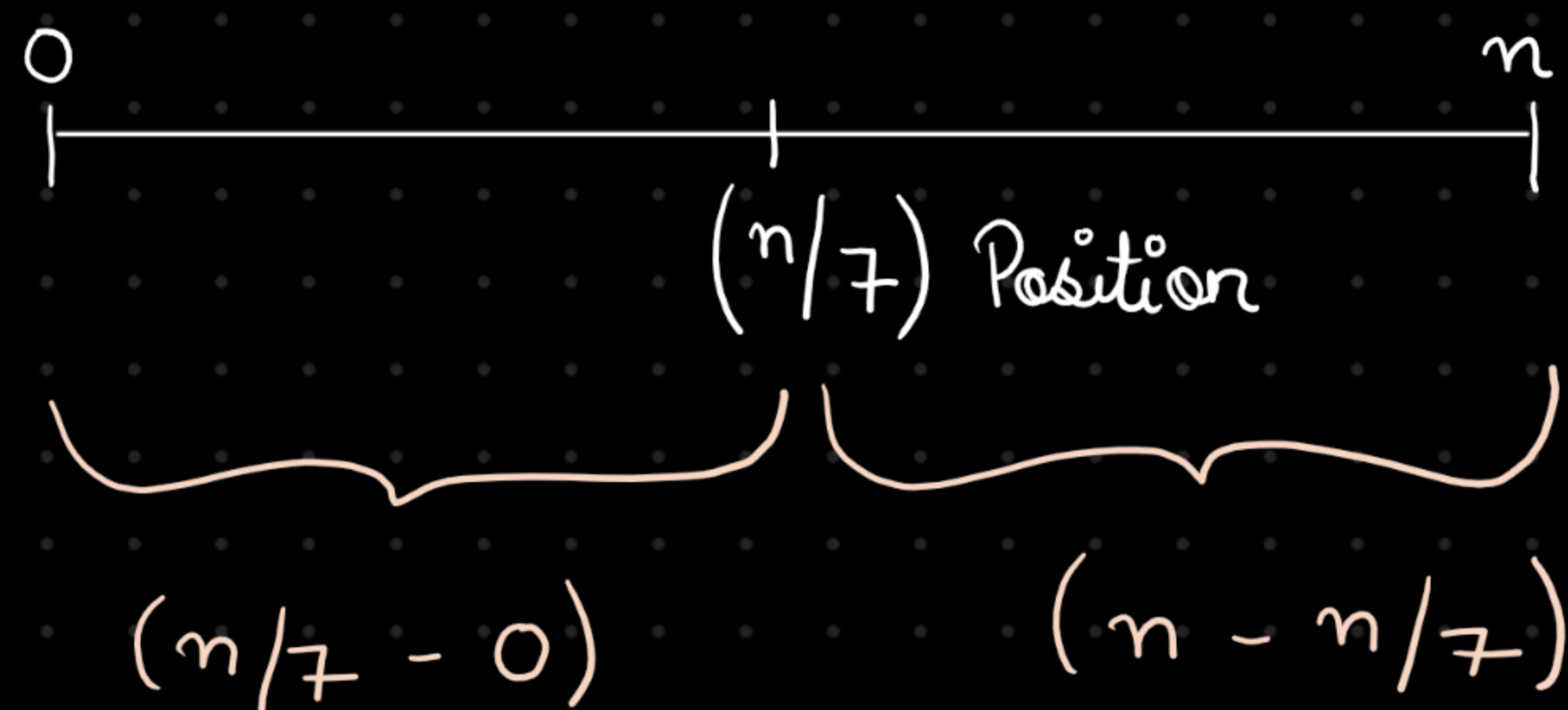
A better Quicksort Method

↳ Randomized Quicksort
↙

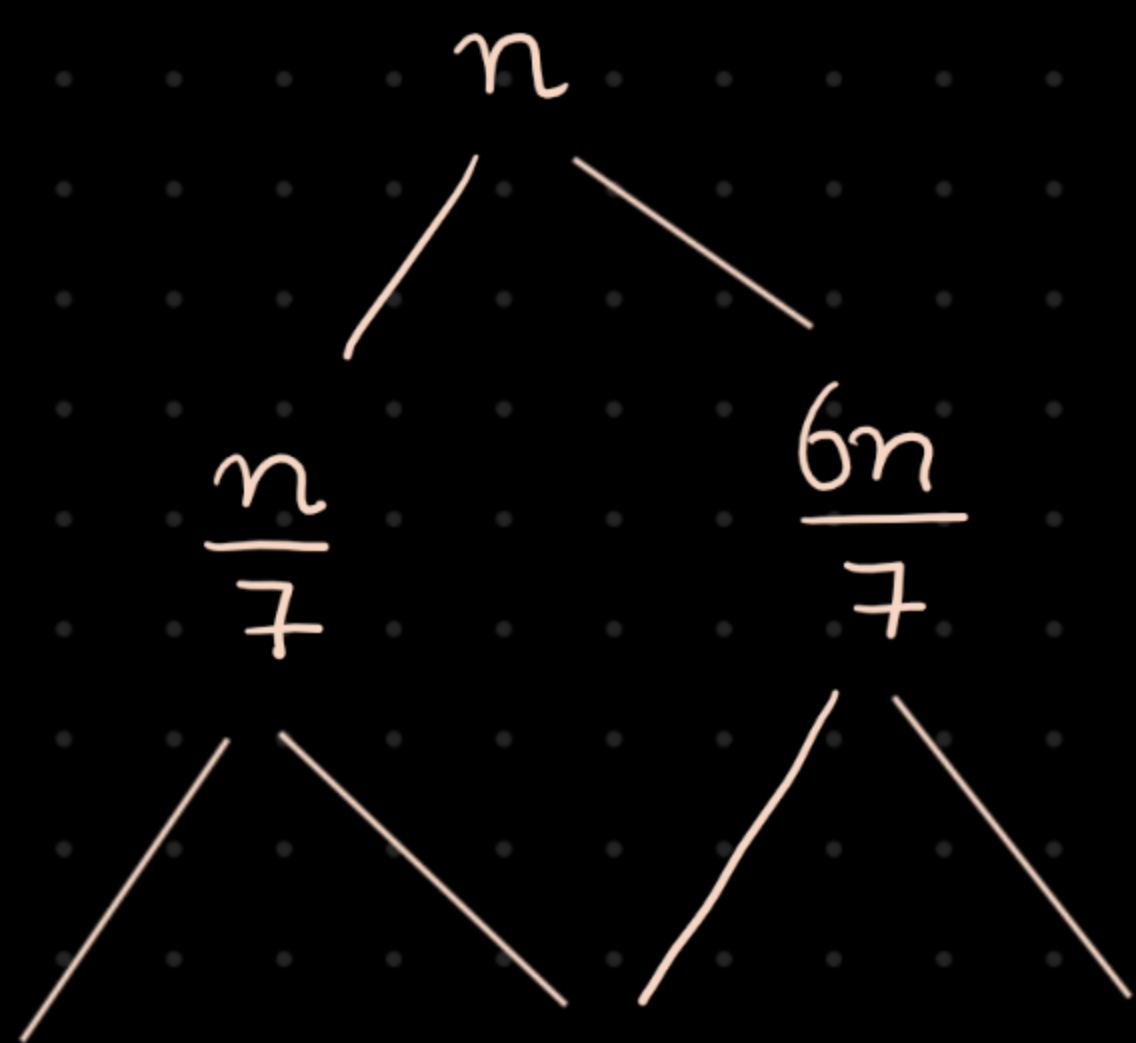
Choose pivot element randomly to decrease the probability of worst case scenario.

Interview

- 1) $n/7^{\text{th}}$ smallest element $\rightarrow$ Quicksort + Partition
 $\hookrightarrow O(n^2)$ $\hookrightarrow O(n)$



$$T(n) = T(n/7) + n(6n/7) + n^2 + n$$



- 2) K^{th} smallest element

(20, 47, 52, 12, 26, 69, 74)

$K = 2$

$\hookrightarrow 2^{\text{nd}}$ smallest element

Selection

Procedure

$\hookrightarrow \text{pivot} + 1 (\text{position})$

selectionProcedure(arr, p, q):

if $K == n$ $\hookrightarrow$ returned by partition Algo

elif $k < m$:

selectionProcedure(arr, p, m-2)

else:

selectionProcedure(arr, m, q)

position = index + 1

Recurrence Relation

$$T(n) = T(m-p) + n$$

or $\hookrightarrow$ left side

$$T(n) = T(q-m) + n$$

$\hookrightarrow$ Right side

Best Case / Average Case

$$T(n) = T(n/2) + n$$

$$\Rightarrow \Theta(n)$$

Worst Case Scenario

$$T(n) = T(n-1) + n$$

$$= O(n^2)$$

1. Kth smallest element [Amazon]

Given an array of n-elements and an integer k, find the kth smallest element present in an array.

For example:

arr = [40, 25, 68, 79, 52, 66, 89, 97]

k = 2

result = 40

```
#include <iostream>
using namespace std;
```